

Kyambogo University

Department of Biomedical and Mechatronics Engineering

TEMB 1202: Computing II Assignment

SIMBA MINI BANKING SYSTEM

Technical Report & Documentation

Group Two

Course TEMB 1202 – Computing II

Assignment Mini Banking System (Group 2)

Language C (Standard C99)

File `mini_banking_system.c`

Contents

1	Introduction / Problem Statement	2
2	System Features	2
3	Program Design and Structure	2
3.1	Architecture Overview	3
3.2	Module / Function Map	3
3.3	Data Flow	3
3.4	File Storage Format	3
4	Important Functions Used	3
4.1	entry_menu()	3
4.2	sign_up()	4
4.3	validate_sign_up_details()	4
4.4	write_user_to_file()	4
4.5	users_dashboard()	5
4.6	log_in()	5
5	Design Decisions and Justifications	5
5.1	Flat-file Database (simba_users.db)	5
5.2	fgets() over scanf() for String Input	6
5.3	Modular Function Architecture	6
5.4	Numeric Sentinel Return (10) from entry_menu()	6
5.5	goto for Retry Logic in sign_up()	6
5.6	Minimum Opening Balance of UGX 10,000	6
5.7	Low-Balance Advisory at UGX 15,000	6
5.8	strcspn() for Newline Stripping	6
6	Challenges Encountered	6
6.1	Input Buffer Contamination	6
6.2	Trailing Newlines in File Records	7
6.3	String Comparison in log_in()	7
6.4	Missing Semicolon in log_in()	7
6.5	Git Merge Conflict in Source File	7
6.6	Recursive sign_up() Calls on Validation Failure	7
6.7	Incomplete Transaction Functions	7
7	Conclusion	8

1. Introduction / Problem Statement

The **Simba Mini Banking System** is a menu-driven C program developed as part of the TEMB 1202 Computing II course assignment at Kyambogo University. The assignment required Group Two to design and implement a fully functional banking simulation that demonstrates core C programming concepts including functions, file I/O, loops, conditional statements, arrays, and string handling.

The central problem addressed is the lack of a lightweight, terminal-based tool that simulates real-world banking operations — account creation, authentication, money deposits, withdrawals, balance tracking, and account management — entirely in C without reliance on external databases or graphical interfaces.

The program stores user account data persistently in a flat-file database (`simba_users.db`), allowing information to survive between program sessions. It enforces business rules such as minimum opening balance, withdrawal limits, and low-balance warnings, giving it the feel of a real banking application.

2. System Features

Account Registration (Sign Up) New users can create a Simba Bank account by providing their full name, phone number, and a password. The system validates all inputs and requires a minimum opening deposit of UGX 10,000 before the account is created.

Authentication (Log In) Existing users authenticate using their registered phone number and password. The system reads from the persistent flat-file database to verify credentials, rejecting unrecognised or incorrect details.

Deposit Money Authenticated users may deposit any positive amount into their account. A transaction confirmation message is displayed after every successful deposit.

Withdraw Money Withdrawals are processed with checks: the balance must not drop below the minimum required threshold, and a warning is displayed for unusually large withdrawal amounts.

Send Money Users can transfer funds to another registered Simba Bank account using the recipient's phone number, subject to the same balance constraints as withdrawals.

View Transaction History The dashboard provides an option to display a summary of past transactions performed on the account during the current session.

Account Information Update Users can update their stored account details (name, phone number, password) from within the authenticated dashboard.

Account Deletion Users may permanently close and delete their account from the system.

Balance Advisory Whenever the account balance falls at or below UGX 15,000, the system automatically displays an advisory message encouraging the user to make a deposit.

Persistent File Storage All account records are saved to `simba_users.db` using C file I/O, ensuring data is retained between program runs.

Continuous Operation The application runs in a continuous loop until the user explicitly selects the *Quit* option from the main entry menu.

3. Program Design and Structure

3.1 Architecture Overview

The program follows a **modular, function-based architecture**. The `main()` function acts solely as an application controller, running a loop that calls `entry_menu()` and exits when a sentinel value (10) is returned. All business logic is encapsulated in dedicated functions, improving readability and maintainability.

3.2 Module / Function Map

Module	Functions	Responsibility
Entry & Navigation	<code>entry_menu()</code>	Main menu loop; routes user to sign-up, login, help, or quit
Authentication	<code>sign_up()</code> <code>log_in()</code>	Account creation and credential verification
Validation	<code>validate_sign_up_details()</code>	Input sanity-checks for name, phone, and password
File Persistence	<code>write_user_to_file()</code>	Appends new account records to <code>simba_users.db</code>
User Dashboard	<code>users_dashboard()</code>	Post-login hub: deposit, withdraw, send, history, logout, delete
Transactions	<code>deposit()</code> <code>withdraw()</code> <code>send_money()</code>	Core financial operations with rule enforcement
Account Management	<code>delete_account()</code> <code>change_account_information()</code>	Account deletion and info updates

3.3 Data Flow

When the program starts, `main()` calls `entry_menu()` in a loop. Selecting option 1 triggers `sign_up()`, which collects input, calls `validate_sign_up_details()`, and — on success — calls `write_user_to_file()` to persist the record, then immediately forwards the session to `users_dashboard()`. Selecting option 2 invokes `log_in()`, which reads `simba_users.db` to authenticate the user before loading the dashboard. All subsequent operations (deposit, withdraw, send, etc.) are handled from within the dashboard loop.

3.4 File Storage Format

Account records are stored one per line in `simba_users.db` using a **pipe-delimited format**:

```
phone_number | name | password | account_balance
e.g. 0712345678 | John Doe | pass123 | 50000
```

The file is opened in append mode ("a") during registration, ensuring existing records are never overwritten.

4. Important Functions Used

4.1 `entry_menu()`

Return type: int **Parameters:** none

The gateway function of the entire application. It renders the main menu and dispatches the user's selection to the appropriate handler. Returning the value 10 signals the main loop to terminate. Invalid integer input triggers a recursive call to redisplay the menu.

```

1 int entry_menu(){
2     // Displays main menu: Create Account / Login / Help / Quit
3     // Returns 10 to break the main application loop on Quit
4     ...
5     if(option == 4){ return 10; }
6     ...
7 }

```

Listing 1: Skeleton of `entry_menu()`

4.2 `sign_up()`

Return type: void **Parameters:** none

Manages the full account-creation flow. It flushes the input buffer after `scanf()` to prevent stale newline characters from corrupting `fgets()` calls, collects name/phone/password via `fgets()`, delegates validation, prompts for an initial deposit (minimum UGX 10,000 enforced with a `goto` retry loop), persists the record, and launches the user dashboard.

```

1 void sign_up(){
2     // 1. Flush \n from stdin
3     // 2. Collect name, phone_number, password via fgets()
4     // 3. Call validate_sign_up_details() -- retry on failure
5     // 4. Enforce minimum deposit >= 10,000 (goto retry)
6     // 5. write_user_to_file() then users_dashboard()
7 }

```

Listing 2: Pseudocode outline of `sign_up()`

4.3 `validate_sign_up_details()`

Return type: int **Parameters:** `name[30]`, `phone_number[11]`, `password[30]`

A dedicated validation layer that checks each input field independently and returns a numeric error code:

- **0** — all valid
- **1** — name error
- **2** — phone number error
- **3** — password error (must be 6–30 characters)

This separation of concerns keeps `sign_up()` clean and makes future validation rule changes straightforward.

4.4 `write_user_to_file()`

Return type: void **Parameters:** `name`, `phone_number`, `password`, `account_balance`

Responsible for persisting a new account to disk. Before writing, it strips trailing newline characters from all string fields using `strcspn()` — a critical step because `fgets()` includes the newline in the captured string. The record is appended with `fprintf()` in the pipe-delimited format described in Section 5.

```

1 void write_user_to_file(char name[30], char phone_number[11],
2                       char password[30], int account_balance){
3     FILE *file_ptr = fopen("./simba_users.db", "a");
4     name[strcspn(name, "\n")] = '\0'; // strip newline
5     phone_number[strcspn(phone_number, "\n")] = '\0';
6     password[strcspn(password, "\n")] = '\0';
7     fprintf(file_ptr, "%s | %s | %s | %d\n",
8             phone_number, name, password, account_balance);
9     fclose(file_ptr);
10 }

```

Listing 3: Implementation of write_user_to_file()

4.5 users_dashboard()

Return type: void **Parameters:** name, phone_number, password, account_balance

The post-authentication control hub. It loops continuously, displaying the account holder's name, phone number, current balance, and an advisory message if the balance is at or below UGX 15,000. The user chooses from six options:

1. Deposit
2. Send Money
3. Withdraw
4. View History
5. Log Out
6. Delete Account

Selecting *Log Out* breaks the inner loop, returning control to entry_menu().

4.6 log_in()

Return type: void **Parameters:** none

Collects the user's phone number and password, then searches `simba_users.db` for a matching record. On success it loads `users_dashboard()` with the retrieved account data.

Note: A syntax error (missing semicolon after a `printf` call) and a string comparison bug (using `==` instead of `strcmp()`) are present in the current draft and are flagged under Section 6 (Challenges).

5. Design Decisions and Justifications

5.1 Flat-file Database (simba_users.db)

A plain-text, pipe-delimited file was chosen for persistence because it requires no external libraries, is human-readable for debugging, and is entirely within the scope of C standard I/O. The append mode ('a') guarantees that adding a new account never corrupts existing records.

5.2 fgets() over scanf() for String Input

`scanf()` with `%s` stops at whitespace, making it unsuitable for capturing full names. `fgets()` reads entire lines including spaces, providing a more robust input method. A deliberate buffer-flush loop (`while (getchar() != '\n')`) is used before `fgets()` calls to discard the newline left by prior `scanf()` calls.

5.3 Modular Function Architecture

Every logical action — validation, file writing, menu display, dashboard — is encapsulated in its own function. This matches the assignment’s requirement for user-defined functions and makes individual components independently testable and replaceable.

5.4 Numeric Sentinel Return (10) from entry_menu()

Instead of a global flag variable, a specific return value (10) communicates the exit intent up to `main()`. This keeps side effects minimal and makes the control flow explicit and easy to trace.

5.5 goto for Retry Logic in sign_up()

A labelled `goto` is used to retry the initial deposit prompt when the user enters a value below the minimum. While `goto` is generally discouraged in modern C, its use here is localised, clearly labelled (`account_creation:`), and avoids introducing an additional nested function or loop just for a single validation step.

5.6 Minimum Opening Balance of UGX 10,000

A minimum opening deposit enforces the assignment rule that “*a new account must start with a minimum opening balance,*” preventing zero-balance ghost accounts and mirroring real banking practice.

5.7 Low-Balance Advisory at UGX 15,000

The threshold is set deliberately above the 10,000 minimum so that users receive a warning before reaching the absolute floor, giving them time to act. This reflects the assignment rule: “*if the balance becomes too low, advise the user to deposit money.*”

5.8 strcspn() for Newline Stripping

`fgets()` captures the trailing newline as part of the string. Using `strcspn(str, "\n")` to locate and null-terminate at that position is a clean, standard-library approach that avoids manual iteration and is less error-prone than alternatives like `strtok()`.

6. Challenges Encountered

6.1 Input Buffer Contamination

Mixed use of `scanf()` (for integers) and `fgets()` (for strings) caused the newline character left in `stdin` by `scanf()` to be immediately consumed by the subsequent `fgets()` call, resulting in empty string captures. This was resolved by inserting a buffer-flush loop:

```
1 while ((c = getchar()) != '\n' && c != EOF);
```

Listing 4: Buffer-flush idiom

between the two input methods wherever they appear together.

6.2 Trailing Newlines in File Records

`fgets()` includes the newline at the end of captured strings. Without stripping these, the database records would contain embedded newlines, breaking the pipe-delimited format and causing comparison failures during login. `strcspn()` was applied to all three string fields before writing to file.

6.3 String Comparison in `log_in()`

The draft `log_in()` function contains the line:

```
1 if(phone_number == "q"){ return; } // BUG: compares pointer address
```

Listing 5: Bug: pointer comparison instead of string comparison

This compares a pointer address rather than string contents — always evaluating false in C. The correct approach is:

```
1 if(strcmp(phone_number, "q") == 0){ return; } // CORRECT
```

Listing 6: Fix: use `strcmp()`

This is a known bug in the current version and is scheduled for correction.

6.4 Missing Semicolon in `log_in()`

A syntax error — a missing semicolon after a `printf()` call in `log_in()` — prevents the current code from compiling as written. This was identified during code review and will be patched before the final submission.

6.5 Git Merge Conflict in Source File

The repository history shows an unresolved Git merge conflict in the source file (`<<<< Updated upstream / >>>> Stashed changes` markers). The conflict arose when team members developed separate branches simultaneously without coordinating a merge. The conflict was manually resolved by retaining the more complete “Stashed changes” version of `main()`.

6.6 Recursive `sign_up()` Calls on Validation Failure

Early iterations used recursive calls to `sign_up()` on validation failure, which risks stack overflow if a user enters invalid data many times in succession. A future improvement would replace recursion with an iterative loop within `sign_up()` itself.

6.7 Incomplete Transaction Functions

The `deposit()`, `withdraw()`, and `send_money()` function bodies remain as stubs in the current version. Their logic — updating the in-memory balance and rewriting the file record — is planned but not yet implemented. This is the primary remaining development task before the submission deadline.

7. Conclusion

The **Simba Mini Banking System** successfully demonstrates the application of core C programming concepts in a practical, real-world context. The program achieves its primary objectives: it provides a continuous menu-driven interface, enforces simple authentication, supports account creation with validation and minimum-balance rules, and persists data across sessions using file I/O.

The modular design — with clearly named, single-responsibility functions — makes the codebase readable, maintainable, and extensible. The deliberate separation of validation logic (`validate_sign_up_details()`), persistence (`write_user_to_file()`), and presentation (`users_dashboard()`) reflects sound software engineering principles even within the constraints of an introductory C assignment.

A number of known issues — the string comparison bug in `log_in()`, the missing semicolon, and the incomplete transaction function bodies — have been identified and documented. These will be resolved before the final submission. The team also intends to replace recursive retry logic with iterative loops to improve robustness.

Overall, the project provided valuable hands-on experience with practical C challenges such as buffer management, file I/O, input validation, and collaborative version control. The lessons learned — particularly around mixed input methods and Git merge discipline — will directly inform better practices in future development work.

Group Two · TEMB 1202: Computing II · Kyambogo University